

Competitive Programming

Section 41: Euler Tour, LCA, and Binary Lifting

Competitive Programming Course

2026-07-10

Table of contents

1	Tree paths by powers of two	3
1.1	LCA algorithm	3
1.2	Path aggregates	4
1.3	K-th node on a path	4
1.4	Minimal C++ lifting helper	4
1.5	Minimal Python lifting helper	4
1.6	Worked LCA trace	5
2	Exercises	6
3	A complete lifting trace	7
4	Three path queries from one primitive	8
5	Constraints, implementation, and tests	9

1 Tree paths by powers of two

Lowest common ancestor turns a path query into two root paths. Store $up[k][v]$, the 2^k -th ancestor of v , and combine values while lifting. This section extends Section 38's ancestor jumps to full path queries.

```
distance(u,v) = depth[u] + depth[v] - 2*depth[lca(u,v)]
```

The preprocessing root can be any fixed node, usually 1. A DFS/BFS records three pieces of information:

- $depth[v]$: number of edges from the root to v ;
- $up[0][v]$: the parent of v ;
- optional edge aggregate from v to $up[0][v]$, such as edge maximum.

Then the jump table is built by doubling:

```
up[k][v] = up[k-1][ up[k-1][v] ]
```

If $best[k][v]$ stores the maximum edge on the upward path of length 2^k starting at v , then:

```
best[k][v] = max(best[k-1][v], best[k-1][up[k-1][v]])
```

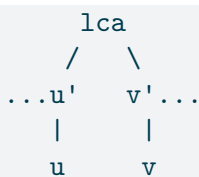
This is the same table shape, with a value carried alongside the ancestor.

1.1 LCA algorithm

To compute $lca(u, v)$:

1. Make u the deeper vertex.
2. Lift u upward until $depth[u] == depth[v]$.
3. If $u == v$, this is the LCA.
4. From the largest power down to zero, if $up[k][u] != up[k][v]$, lift both.
5. Now the two vertices are children of the LCA, so return $up[0][u]$.

Diagram:



The simultaneous lifting step keeps u and v strictly below the LCA until the last move. This is why we compare ancestors before jumping.

1.2 Path aggregates

For maximum edge on path $u-v$, do the same lifting but update an answer whenever you jump:

```
ans = max(ans, best[k][u])
u = up[k][u]
```

You do this while equalizing depths and while lifting both sides. The path is partitioned into disjoint upward chunks, so every edge is included exactly once.

For maximum edge on a path, lift the deeper endpoint upward while taking maximums from the same jump table. For the k -th node on a path, split the path at the LCA: first move upward from u , otherwise move upward from v by the remaining distance from the far end.

1.3 K-th node on a path

Let $w = \text{lca}(u, v)$.

```
u-side length = depth[u] - depth[w] + 1
total nodes    = depth[u] + depth[v] - 2*depth[w] + 1
```

If k is inside the u side, lift u by $k-1$. Otherwise the answer lies on the v side. Convert it to a distance upward from v :

```
need_from_v = total_nodes - k
answer = ancestor(v, need_from_v)
```

The most common bug is off-by-one: the first node on the path is u , so queries usually use 1-based k .

1.4 Minimal C++ lifting helper

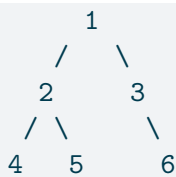
```
int jump(int v, int d) {
    for (int k = 0; d; k++, d >>= 1) {
        if (d & 1) v = up[k][v];
    }
    return v;
}
```

1.5 Minimal Python lifting helper

```
def jump(v, d):
    k = 0
    while d:
        if d & 1:
            v = up[k][v]
        d >>= 1
        k += 1
    return v
```

1.6 Worked LCA trace

Consider:



Query $\text{lca}(4,5)$: both depths are equal. Their immediate parents are both 2, so after the simultaneous lifting loop the answer is 2.

Query $\text{lca}(4,6)$: both depths are equal, but parents differ:

```
parent(4)=2, parent(6)=3
```

Lifting both by 1 gives nodes 2 and 3; their parents match at 1, so the answer is 1.

When edge weights are present, the same jumps collect maximums along the lifted chunks. Always update the aggregate before overwriting the vertex.

2 Exercises

3 A complete lifting trace

Use the parent list from the local sample: `parent[2]=1, parent[3]=1, parent[4]=2, parent[5]=2`. Depths are 0,1,1,2,2. The binary table records `up[j][v]`, the ancestor 2^j edges above v ; the root points to itself.

<code>v</code>	1	2	3	4	5
<code>up[0]</code>	1	1	1	2	2
<code>up[1]</code>	1	1	1	1	1
<code>depth</code>	0	1	1	2	2

To find `lca(4,5)`, depths are equal. At bit one both ancestors are one, so do not jump; at bit zero the parents are equal, so their common parent two is the LCA. For distance, use `depth[4]+depth[5]-2*depth[2]=2`. For `lca(3,4)`, lift four by one to two, then the parents are both one.

The invariant for `up[j][v]` is exact: it is the ancestor reached after 2^j parent edges. The recurrence `up[j][v]=up[j-1][up[j-1][v]]` composes two correct half-jumps, proving the table by induction. Descending bits only jumps when the two nodes remain below their LCA; the final parents are therefore the lowest common ancestor.

4 Three path queries from one primitive

Distance. Once $c = \text{lca}(u, v)$, the path has the two disjoint upward pieces $u \rightarrow c$ and $v \rightarrow c$, so its edge count is $\text{depth}[u] + \text{depth}[v] - 2 * \text{depth}[c]$.

Maximum edge. Store $\text{mx}[j][v]$, the maximum edge on the same 2^j upward segment. When a lift uses bit j , merge $\text{mx}[j][v]$ into the answer before moving v . Its recurrence uses max over the two halves. For $u=v$, the local answer is zero because the path contains no edge.

K-th node. Let $\text{left} = \text{depth}[u] - \text{depth}[c] + 1$ and $\text{total} = \text{depth}[u] + \text{depth}[v] - 2 * \text{depth}[c] + 1$. If $k \leq \text{left}$, lift u by $k-1$. Otherwise the requested vertex is $\text{total} - k$ edges above v ; this reverse lift avoids constructing a path vector. Reject $k < 1$ or $k > \text{total}$ first.

5 Constraints, implementation, and tests

For n, q near $2e5$, a parent climb per query is a chain-length disaster; $LOG = \lceil \log_2(n+1) \rceil$ gives $O(n \log n)$ preprocessing and $O(\log n)$ query time. The local parent order makes depths available while reading. In an arbitrary edge-list tree, build adjacency and perform iterative DFS/BFS first.

C++. Allocate `vector<vector<int>> up(LOG, vector<int>(n+1))`; use `long long` for weighted aggregates if weights can sum, and initialize the empty maximum as zero only because local weights are nonnegative.

Python. Avoid recursive DFS on a long chain. Use lists indexed $0..n$ and an iterative stack/queue; `LOG = (n+1).bit_length()` is clear and safe.

Mistake	Test
confuse nodes and edges in distance	<code>u==v</code> must output zero
omit equal-depth lift	ancestor/descendant query
use a one-based k as zero-based	first and last node on a path
aggregate after moving	one heavy edge immediately above a node

Run `CP_TARGET=solution python3 sections/41_euler_tour_lca_binary_lifting/check.py`. For trees of at most 20 nodes, a parent-walk oracle can list each path and check distance, maximum edge, and every valid or invalid k directly.

- A. Tree Distances: Given a rooted tree and queries $u \ v$, print the number of edges on the path.
- B. Max Edge On Path: Each edge to a parent has a weight. For each $u \ v$, print the maximum edge weight on the path.
- C. Kth Node On Path: For query $u \ v \ k$, print the k -th node on the path from u to v , or -1 .